AI GEN X

Training & Architecting Autonomous Intelligence

Agentic Architecture & Framework Document — Part 2

Sections 9–17: PPM & Editor Agents, Cross-Agent Handoffs, Data Flow, HITL Gates, Security, Deployment, Demo Mode & Observability

BATCH 2 OF 6 — PART 2 — CONFIDENTIAL

Prepared for: Dragonfruit Media · Brendan, Yazan, Emily

Prepared by: AI Gen X · March 2026

9. Agent Detail: Post-Production Manager (PPM) Agent (Continued)

> **Time Audit Context:** PPMs logged 242.5h against 200h capacity (121% overloaded). Dream: Reviews 35%, Editor Management 30%, Hiring+Onboarding 15%. The PPM Agent eliminates reactive overhead (Slack 14% → 3%, ClickUp 5% → 1%) so PPMs shift from reactive troubleshooting to intentional quality coaching.

9.2 Key Tools (Continued)

ED-1: Automated Video QC Checker

```
@function_tool
async def run_qc_check(
    video_url: str,          # Frame.io asset URL or local path
    client_name: str,        # For client-specific checklist
    content_type: str,       # "longform" | "shortform"
) -> dict:
    """Reviews rendered video against client-specific QC checklist.
    Produces timestamped error report with severity ratings.

    Checklist items: intro/outro branding, lower-third accuracy, audio levels,
    color grading consistency, b-roll placements, caption accuracy, export
settings.

    Maps to: ED-1 (Automated Video QC Checker) — HIGH priority
    Dream impact: Slack 14% → 3% (PPMs). Automated QA notifications eliminate manual
pings.
    Pipeline steps: Replaces manual QC review cycle that causes revision loops.

    Integration: Frame.io API (get asset) → Video analysis model → ClickUp MCP
(create QC task if issues found)

    Output: {
        pass: bool,
        overall_score: float,
        issues: [{
            timestamp: str,
            checklist_item: str,
            severity: "critical" | "warning" | "info",
            description: str,
            screenshot_url: str
        }],
        checklist_results: [{item: str, status: "pass" | "fail" | "warning"}]
    }
    """
```

ED-5: Frame.io Comment Sentiment Analyzer

```
@function_tool
async def analyze_frameio_sentiment(
    project_id: str,         # Frame.io project ID
    client_name: str,
    timeframe_days: int = 30,
) -> dict:
    """Monitors Frame.io review comments. Produces:
    1. Video-level sentiment (tone/satisfaction per review)
    2. Client sentiment tracker (patterns over time, escalation risks)

    Runs passively. Alerts when sentiment drops below threshold.

    Maps to: ED-5 (Frame.io Comment Sentiment Analyzer) — MEDIUM priority
    Dream impact: Editor Management 11% → 30% (PPMs). AI flags issues before they
escalate.
    Integration: Frame.io API v2 (get_comments) → NLP sentiment model → Slack MCP
(alert)

    Output: {
        video_sentiments: [{video_name, sentiment_score, key_themes, tone}],
        client_trend: {overall: float, direction: "improving"|"declining"|"stable"},
        alerts: [{type: str, message: str, severity: str}],
        recurring_complaints: [str]
    }
    """
```

ED-3: Rough Cut Acceleration (Agent-Assisted)

```
@function_tool
async def prepare_rough_cut(
    script_text: str,        # Script or outline
    footage_location: str,   # Dropbox path or Frame.io project
    client_name: str,
) -> dict:
    """Prepares rough cut assembly by:
    1. Ingesting raw footage alongside script
    2. Auto-syncing dialogue to transcript (via Descript integration)
    3. Removing silences and obvious mistakes
    4. Producing structured timeline organized by script section

    Maps to: ED-3 (Rough Cut Acceleration Tool) — HIGH priority
    Dream impact: Core Editing 66% → 80% (Editors). Multiplier across 60+ editors.
    Integration: Dropbox API (get footage) → Descript API (transcribe + sync) →
Timeline output

    Note: This is the highest-ROI automation opportunity in the organization.
    Even 15 min saved per editor per video × 60 editors × 15 videos/month = 225
hours/month saved.
    """
```

9.3 PPM Agent ReAct Flow: Automated QC Pipeline

Trigger: Editor marks task as "Ready for Review" in ClickUp (status change detected via ClickUp MCP webhook).

▼

Step 1 — REASON: PPM Agent detects status change. Identifies: client=Alpha, editor=Editor 3, content_type=longform, version=V1. Decides: run automated QC before PPM manual review.

▼

Step 2 — ACT: Calls `run_qc_check(video_url=frameio_link, client="Alpha", type="longform")`. QC model analyzes video against Alpha's checklist.

▼

Step 3 — OBSERVE: QC result: 2 warnings (audio level at 3:42, missing lower-third at 7:15), 0 critical issues. Overall score: 87/100.

▼

Step 4 — ACT: Posts QC report to #ppm-qc-reports in Slack:
"🔍 **QC Report: Client Alpha - LF - Ep5 - V1**
Score: 87/100 | ⚠️ 2 warnings | ✅ 0 critical
• 3:42 — Audio level -18dB (spec: -14dB) [WARNING]
• 7:15 — Missing lower-third for guest name [WARNING]
PPM: Review needed. [📋 View Full Report] [✅ Approve Anyway] [↩️ Send Back to Editor]"

▼

Step 5 — HITL: PPM clicks "Send Back to Editor" → Agent creates revision task in ClickUp with QC notes attached, notifies Editor 3 in Slack with specific timestamps and fixes needed.

9.4 Survey Context

From AI Tool Survey: PPMs — 4 respondents. ChatGPT 100%, ElevenLabs 100%, Runway 75%, Gemini 50%, Claude 50%. Use cases: Audio Fixes, SOPs, Video Concepting, Team Support, Script Analysis. *"Using the unpaid version has worse results... GPT for managers would help."* — Mahmoud K.

Insight: PPMs are the audio experts (100% ElevenLabs — only role with universal non-ChatGPT adoption). The PPM Agent leverages their technical sophistication — they'll trust automated QC because they understand the underlying checks. The agent handles the routing/notification overhead (Slack 14% → 3%) so PPMs focus on the reviews they're best at.

10. Agent Detail: Editor Support Agent

> **Time Audit Context:** 54 editors, 2,119.8h logged. Core Editing 66% → Dream 80%. Idle/Waiting 6.5% → 2%. The Editor Support Agent doesn't replace editors — it eliminates the friction around editing: waiting for assignments, downloading footage, looking up assets, and navigating ClickUp.

10.1 Agent Configuration

Property	Value
Name	Editor Support Agent
Model	gpt-4o-mini (lighter model — high volume, simpler tasks)
Pipeline Phases	Post-Production
Wish List Tools	ED-3 (Rough Cut Acceleration), ED-4 (Shortform Acceleration)
MCP Servers	ClickUp MCP (task lookup), Slack MCP (notifications)
Function Tools	descript_ingest_trigger, asset_locator, export_validator, shortform_clip_selector
Handoffs	→ PPM Agent (QC escalation), → PC Agent (assignment questions)
HITL Gates	Minimal — mostly read operations and triggers. HITL on: export publish, footage download initiation.

10.2 Key Tools

Automated Descript Ingest Pipeline

```
@function_tool
async def trigger_descript_ingest(
    footage_dropbox_path: str,   # Dropbox path to raw footage
    project_name: str,
    editor_name: str,
) -> dict:
    """Automates the footage → Descript pipeline:
    1. Detects new footage in Dropbox (via webhook or polling)
    2. Downloads to processing server
    3. Uploads to Descript for transcription
    4. Notifies editor when transcript is ready

    Dream impact: Footage Downloading 2.7% → 1% (Editors). Automated file ingest.
```

```
    Integration: Dropbox API → Processing → Descript API → Slack MCP (notify
editor)
    """
```

ED-4: Shortform Clip Selector

```
@function_tool
async def select_shortform_clips(
    longform_transcript: str,     # Full transcript of longform video
    client_style: str,            # Client's shortform style preferences
    num_clips: int = 5,           # Number of clips to suggest
) -> dict:
    """AI-powered clip selection from longform source material.
    Identifies high-engagement moments for Reels/Shorts/TikToks.

    Maps to: ED-4 (Shortform Edit Acceleration Tool) — HIGH priority
    Dream impact: Core Editing 66% → 80%. Dedicated shortform pipeline reduces per-
unit edit time.

    Output: {
        suggested_clips: [{
            start_time: str, end_time: str, duration: float,
            transcript_excerpt: str, engagement_score: float,
            suggested_caption: str, music_mood: str
        }]
    }
    """
```

10.3 Editor Agent ReAct Flow: Assignment Notification

Trigger: PC Agent assigns a new task to Editor 3 via ClickUp MCP.

▼

Step 1: Editor Support Agent detects assignment. Looks up task details: Client Alpha, LF, Ep5, footage in Dropbox at /Alpha/Ep5/raw/.

▼

Step 2: Calls `trigger_descript_ingest(path="/Alpha/Ep5/raw/", project="Alpha-Ep5", editor="Editor 3")`. Footage begins processing.

▼

Step 3: Sends Slack DM to Editor 3: "📹 **New Assignment:** Client Alpha - LF - Ep5. Script: [Notion link]. Footage: processing in Descript (ETA: 15 min). Style guide: [link]. Due: Thursday. Questions? Reply here or ask @PPM-Agent."

10.4 Survey Context

From AI Tool Survey: Editors — 46 respondents (63% of team). ChatGPT 89%, Gemini 67%, Freepik 57%, ElevenLabs 52%, Firefly 50%. Power users averaging 5+ tools each. 17 pay out of pocket for ChatGPT. *"ChatGPT feels like my art director."* — Levi C.

Insight: Editors are already the most AI-savvy role. They don't need an agent to do their creative work — they need an agent to eliminate the 6.5% idle time (waiting for assignments, footage, feedback) and the 2.7% footage downloading overhead. The Editor Support Agent is a logistics agent, not a creative agent.

11. Cross-Agent Handoff Architecture

DFM's pipeline is inherently cross-functional. A single client request often touches multiple roles. The handoff architecture defines how agents delegate to each other while maintaining context and traceability.

11.1 Handoff Map

From Agent	To Agent	Trigger	Context Passed	Example
Orchestrator	Any role agent	Intent classification	User message + classified intent	"Create a task" → Producer Agent
Producer	PC Agent	Task needs assignment	Task ID, pod, content type, deadline	Producer creates batch → PC assigns editors
Producer	CD Agent	Creative brief needed	Client name, content type, batch details	New batch → CD starts ideation
PC Agent	PPM Agent	Task ready for review	Task ID, editor, video link, version	Editor submits V1 → PPM reviews
PC Agent	Editor Support	Task assigned to editor	Task ID, editor, footage location, script link	Assignment → Editor gets onboarding DM
CD Agent	PPM Agent	Creative review needed	Script, retention score, packaging brief	CD finalizes script → PPM preps editor brief
PPM Agent	PC Agent	Reassignment needed	Task ID, reason, suggested editor	Editor overloaded → PC reassigns
PPM Agent	CD Agent	Creative escalation	Video link, QC report, client feedback	V2 still off-brand → CD reviews

11.2 Handoff Implementation (OpenAI Agents SDK)

```
# Handoffs use the SDK's agent-as-tool pattern.
# When Producer Agent needs PC to assign editors, it "calls" the PC Agent as a
tool.

producer_agent = Agent(
    name="Producer Agent",
    instructions="...",
    tools=[...],
    handoffs=[
        Handoff(
            agent=pc_agent,
            description="Delegate to PC Agent for task assignment, status updates,
and publishing prep.",
```

```
            context_filter=lambda ctx: {
                "task_id": ctx.get("task_id"),
                "pod_name": ctx.get("pod_name"),
                "content_type": ctx.get("content_type"),
                "deadline": ctx.get("deadline"),
            }
        ),
        Handoff(
            agent=cd_agent,
            description="Delegate to CD Agent for creative briefs, ideation, and
script work.",
        ),
    ]
)

# The handoff is visible in the trace:
# [Producer Agent] → Reasoning: "Batch created. Now need to assign editors."
# [Producer Agent] → Handoff to PC Agent with context: {task_id: "abc", pod:
"Mango"}
# [PC Agent] → Reasoning: "Received assignment request. Checking editor
capacity..."
# [PC Agent] → Tool call: capacity_based_assigner(...)
# [PC Agent] → Result returned to Producer Agent
```

11.3 End-to-End Handoff Example: New Client Batch

User (Producer): "Set up a new batch for Mango Pod: 4 longform videos for Client Alpha, due March 21"

▼

Orchestrator → Producer Agent: Intent = batch creation. Routes to Producer.

▼

Producer Agent: Creates 4 ClickUp tasks (after HITL approval). Then handoffs to PC Agent: "Assign editors to these 4 tasks."

▼

PC Agent: Runs capacity algorithm. Assigns Editor 1 (×2) and Editor 3 (×2). HITL approval. Then handoffs to Editor Support Agent: "Notify editors and start footage ingest."

▼

Editor Support Agent: Sends assignment DMs to Editor 1 and Editor 3. Triggers Descript ingest for footage. Posts confirmation to #mango-pod channel.

▼

Producer Agent (receives result): Posts summary to Producer: "✅ Batch set up. 4 tasks created, 2 editors assigned, footage ingesting. Timeline: rough cuts by March 17, V1 by March 19, delivery March 21."

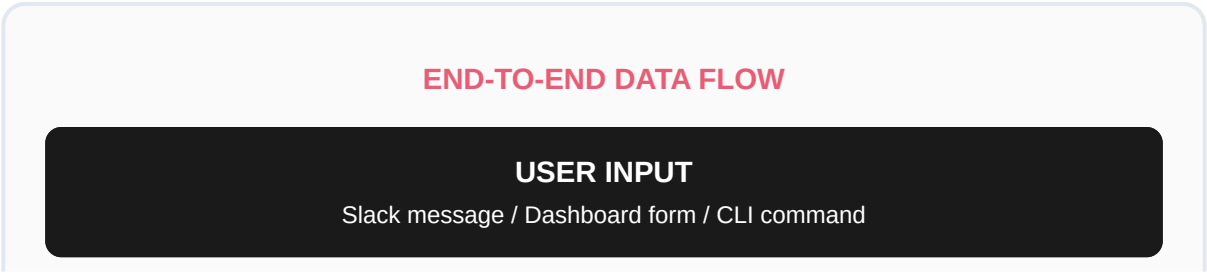
Total time: ~2 minutes (with HITL approvals). Current manual process: 45-60 minutes across Producer + PC.

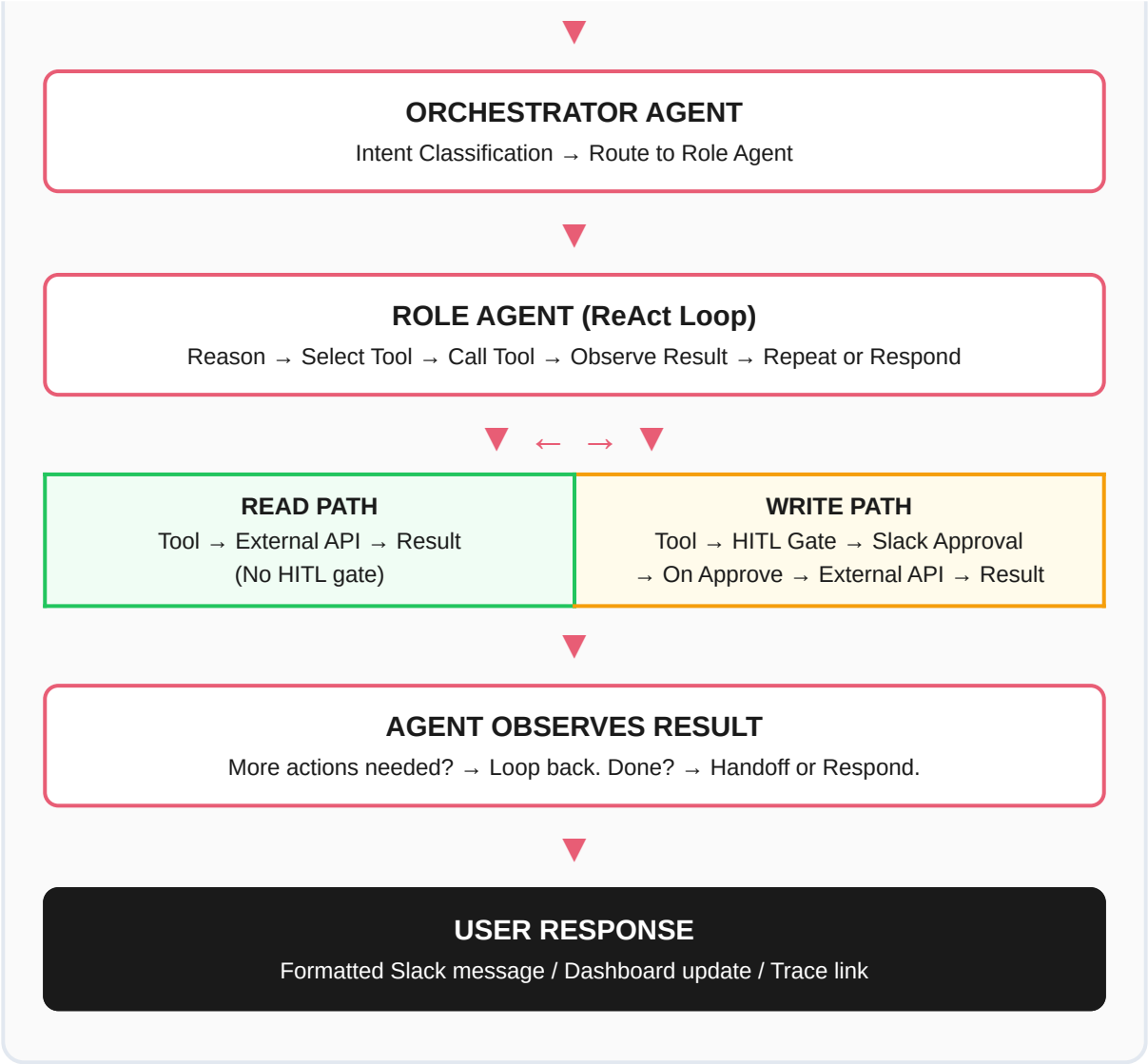
12. Data Flow — End-to-End Request Lifecycle

12.1 Request Lifecycle Stages

Stage	What Happens	Layer	Data
1. Input	User sends message via Slack, Dashboard, or CLI	Layer 1 (UI)	Natural language text + optional attachments
2. Routing	Orchestrator classifies intent, selects role agent	Layer 2 (Orchestration)	Intent label + confidence score
3. Reasoning	Role agent reads instructions + context, plans action sequence	Layer 2 (Orchestration)	Agent instructions + conversation history + tool descriptions
4. Tool Selection	Agent selects tool(s) to call based on reasoning	Layer 2 → Layer 3	Tool name + typed parameters
5. Tool Execution (Read)	Read operations execute immediately. MCP or Function Tool calls external API.	Layer 3 (Tools)	API request → API response
6. HITL Gate (Write)	Write operations pause. Agent posts approval request to Slack.	Layer 4 (HITL)	Proposed action summary + [Approve/Reject/Edit] buttons
7. Execution (Write)	On approval, tool executes write operation against external system.	Layer 3 → Layer 5	API write request → confirmation
8. Observation	Agent reads tool result, decides if more actions needed (ReAct loop).	Layer 2	Tool result + agent reasoning
9. Handoff (if needed)	Agent delegates to another role agent with context.	Layer 2	Handoff context + conversation state
10. Response	Final result posted to user via Slack or Dashboard.	Layer 1	Formatted response + action confirmation + links

12.2 Data Flow Diagram




ORCHESTRATOR AGENT
Intent Classification → Route to Role Agent
ROLE AGENT (ReAct Loop)
Reason → Select Tool → Call Tool → Observe Result → Repeat or Respond
READ PATH
Tool → External API → Result
(No HITL gate)
WRITE PATH
Tool → HITL Gate → Slack Approval
→ On Approve → External API → Result
AGENT OBSERVES RESULT
More actions needed? → Loop back. Done? → Handoff or Respond.
USER RESPONSE
Formatted Slack message / Dashboard update / Trace link

13. Human-in-the-Loop (HITL) Gate Architecture

HITL gates are the safety mechanism that ensures no agent takes irreversible action without human approval. This directly addresses DFM's requirement: *"Human-in-the-loop review; automated actions flagged in dedicated Slack channel."*

13.1 The 7 Critical HITL Gates

#	Gate Name	Trigger	Approval Channel	Approver	Timeout Action
1	**Task Creation**	Agent calls create_task (ClickUp)	#agent-approvals-tasks	Producer or PC	Auto-reject after 4h
2	**Status Update**	Agent calls update_status (ClickUp)	#agent-approvals-tasks	PC or PPM	Auto-reject after 4h
3	**Message Send**	Agent calls send_message to client channel (Slack)	#agent-approvals-comms	Producer	Auto-reject after 2h
4	**Task Assignment**	Agent calls assign_task (ClickUp)	#agent-approvals-tasks	PC	Auto-reject after 4h
5	**Review Routing**	Agent routes video to CD or client for review	#agent-approvals-reviews	PPM	Auto-reject after 4h
6	**Invoice Action**	Agent triggers invoice generation or reminder	#agent-approvals-finance	Finance lead	Auto-reject after 24h
7	**Escalation**	Agent detects sentiment drop or deadline risk	#agent-approvals-escalation	Producer + Leadership	Auto-escalate after 1h

13.2 HITL Gate Implementation

```
# HITL gates are implemented as a wrapper around write tools.
# The wrapper intercepts the tool call, posts to Slack, and waits for approval.

from agents import function_tool
import asyncio

class HITLGate:
    def __init__(self, gate_name: str, approval_channel: str, timeout_hours: float =
4):
```

```
        self.gate_name = gate_name
        self.channel = approval_channel
        self.timeout = timeout_hours

    async def request_approval(self, action_summary: str, agent_name: str) -> bool:
        """Posts approval request to Slack, waits for response."""
        # 1. Format approval message with action details
        # 2. Post to approval channel with interactive buttons
        # 3. Wait for button click (approve/reject) or timeout
        # 4. Log decision to trace
        # 5. Return True (approved) or False (rejected/timeout)

        message = f"""
        🤖 **{agent_name}** requests approval:

        **Action:** {action_summary}
        **Gate:** {self.gate_name}

        [✅ Approve] [❌ Reject] [✏️ Edit]

        ⏰ Auto-rejects in {self.timeout}h if no response.
        """
        # Post via Slack MCP → wait for interaction → return decision

# Usage in a tool:
@function_tool
async def create_task_with_hitl(task_data: dict) -> dict:
    gate = HITLGate("Task Creation", "#agent-approvals-tasks")
    summary = f"Create task: {task_data['name']} in {task_data['list']}"

    approved = await gate.request_approval(summary, "Producer Agent")
    if approved:
        return await clickup_mcp.create_task(**task_data)
    else:
        return {"status": "rejected", "message": "Task creation rejected by
approver."}
```

13.3 HITL Graduation Path

Phase 1 (Months 1-2): All 7 gates active. Every write operation requires approval.
Phase 2 (Months 3-4): Low-risk gates become auto-approve with notification. Status updates and internal messages auto-execute but post confirmation to Slack.
Phase 3 (Months 5-6): Only high-risk gates remain active: client-facing messages, invoice actions, escalations. Task creation and assignment auto-execute for trusted patterns.
Phase 4 (Month 6+): Configurable per-pod. Each Producer can set their own approval thresholds. Full autonomy for routine operations, HITL for exceptions.

14. Security, Authentication & Error Handling

14.1 Authentication Architecture

System	Auth Method	Token Storage	Rotation
ClickUp MCP	OAuth 2.0 (official MCP supports OAuth)	Encrypted env vars (.env) + secrets manager	Auto-refresh via OAuth flow
Slack MCP	Bot Token (xoxb-)	Encrypted env vars	Manual rotation quarterly
Notion MCP	OAuth 2.0 (hosted MCP)	Encrypted env vars	Auto-refresh via OAuth flow
Frame.io API	API Key / OAuth 2.0	Encrypted env vars	Manual rotation quarterly
YouTube API	API Key + OAuth 2.0	Encrypted env vars	Auto-refresh via OAuth flow
Dropbox API	OAuth 2.0	Encrypted env vars	Auto-refresh
OpenAI API	API Key	Encrypted env vars	Manual rotation quarterly

14.2 Permission Model

Agent	ClickUp Permissions	Slack Permissions	Other Permissions
Producer	Read all spaces, Create/Update tasks in assigned pods	Read/Write all internal channels, Read client channels	Frame.io: Read comments
PC	Read all spaces, Create/Update/Assign tasks	Read/Write internal channels	Dropbox: Read files
CD	Read only (ClickUp → 0% in dream)	Read internal, Write creative channels	Notion: Read/Write client pages. YouTube: Read analytics
PPM	Read tasks, Update status (QC routing)	Read/Write QC channels	Frame.io: Read comments + assets
Editor Support	Read assigned tasks only	Write DMs to editors only	Dropbox: Read footage folders. Descript: Trigger ingest

14.3 Error Handling Strategy

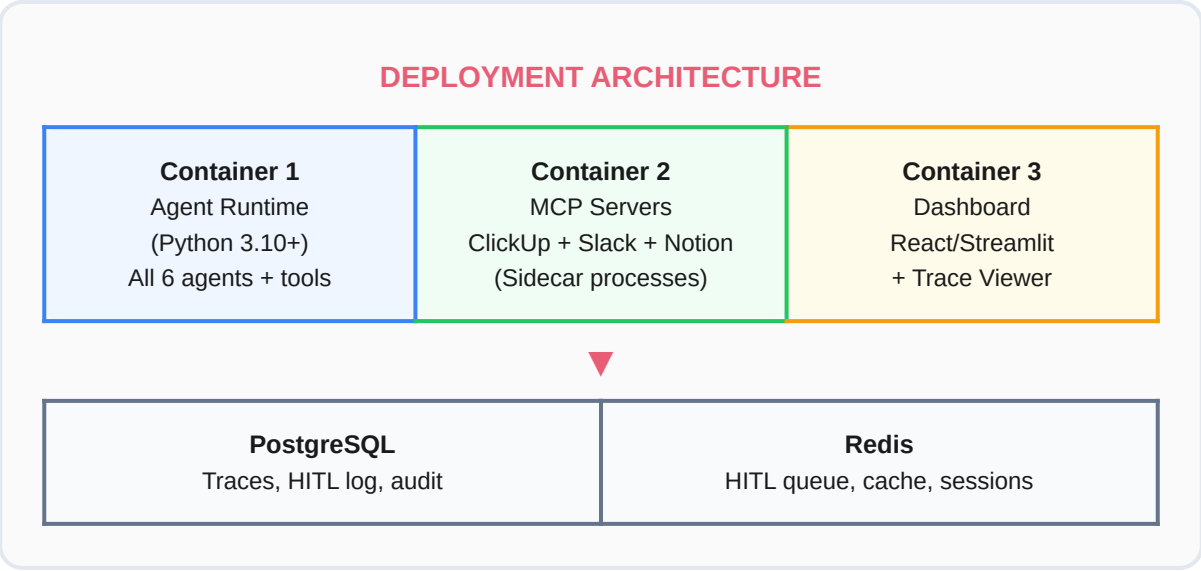
Error Type	Handling	User Impact
API Rate Limit	Exponential backoff with jitter. Max 3 retries. Queue if persistent.	Slight delay. Agent posts "Processing, please wait..." in Slack.
Auth Failure	Attempt token refresh. If fails, alert #agent-ops channel. Fall back to mock mode.	Agent posts "Unable to connect to [system]. Using cached data. Ops team notified."
Tool Execution Error	Agent catches error, reasons about alternative approach. If no alternative, reports to user.	Agent posts error summary with suggested manual action.
HITL Timeout	Auto-reject (or auto-escalate for Gate 7). Log timeout event.	Agent posts "Approval timed out. Action not taken. Resubmit when ready."
Model Hallucination	Guardrails validate tool parameters against schema. Invalid calls rejected before execution.	No user impact — caught before execution.
Cross-Agent Handoff Failure	If target agent fails, control returns to source agent with error context.	Source agent reports: "Handoff to [agent] failed. Completing manually."

15. Infrastructure & Deployment Architecture

15.1 Deployment Stack

Component	Technology	Purpose
Agent Runtime	Python 3.10+ on Docker containers	Runs OpenAI Agents SDK, all agents and tools
MCP Servers	Node.js (ClickUp MCP), Python (Slack, Notion MCP)	Official MCP server processes, managed as sidecars
Dashboard	React (Vite) or Streamlit	User interface for agent interaction and trace viewing
Message Queue	Redis or RabbitMQ	HITL approval queue, async task processing
Database	PostgreSQL	Agent traces, HITL decisions, user preferences, audit log
Hosting	AWS (ECS/Fargate) or Railway	Container orchestration, auto-scaling
Secrets Management	AWS Secrets Manager or .env (dev)	API keys, OAuth tokens, MCP credentials
CI/CD	GitHub Actions	Automated testing, deployment, MCP server updates

15.2 Deployment Diagram



16. Demo Mode & Mock Data Architecture

All connectors support **demo mode**: if API keys are not set, they return mock data from the `mock_data/` directory. This is critical for development, testing, and demos to DFM leadership without touching production systems.

16.1 Mock Data Sources (From DFM Reference Documents)

Mock Data File	Source Document	Contents
`pipeline_stages.json`	DFM Current Pipeline.xlsx	All pipeline phases, steps, roles, tools, automation notes
`time_allocations.json`	DFM Current vs Dream Time Allocations.xlsx	Current %, dream %, key notes per role per task category
`wishlist_tools.json`	DFM AI Tool Wishlist.docx	All 30+ tools: ID, name, role, priority, description, input, output
`ai_survey_summary.json`	DFM AI Tool Survey.pptx	Per role: top tools, use cases, top requests, quotes
`tool_stack.json`	DFM Current Pipeline Tool Stack.xlsx	All active tools, costs, users, subscription details
`pods.json`	Derived from pipeline + brief	Pod names, members, clients, capacity
`phase_roles.json`	Derived from pipeline	Phase → roles mapping for demo navigation
`tool_io.json`	Derived from wishlist	Per tool: input bullets, output bullets
`tool_integration.json`	Defined by architecture	Per tool: MCP vs API, tools used, GIF ID
`tool_dream_mapping.json`	Derived from time allocations + wishlist	Per tool: dream impact, current→dream % shift

16.2 Demo Mode Implementation

```
# Every connector checks for demo mode before making real API calls.

class ClickUpConnector:
    def __init__(self):
        self.api_key = os.getenv("CLICKUP_API_KEY")
        self.demo_mode = self.api_key is None

    async def create_task(self, list_id: str, name: str, **kwargs) -> dict:
        if self.demo_mode:
```

```
            # Return mock response from mock_data/clickup_tasks.json
            return {
                "id": f"mock_{uuid4().hex[:8]}",
                "name": name,
                "status": "Open",
                "url": f"https://app.clickup.com/t/mock_{name.replace(' ', '_')}",
                "_demo": True  # Flag so UI shows "Demo Mode" badge
            }
        else:
            # Real API call via ClickUp MCP or REST API
            return await self._real_create_task(list_id, name, **kwargs)

# The React demo (from REACT_DEMO_PLAN.md) uses this same mock data.
# Pipeline → Phase → Role → Opportunity → Run Demo → mock input/output displayed.
```

16.3 Alignment with React Demo Plan

From REACT_DEMO_PLAN.md: The demo hierarchy is Pipeline → Phase → Role → Opportunity. Each opportunity shows: run demo (mock), GIF (agent trajectory), input/output, integration (MCP vs API), dream allocation mapping.

Architecture alignment: The mock data files listed above directly populate the demo. `phase_roles.json` drives the Phase → Role navigation. `tool_io.json` provides input/output for each opportunity. `tool_integration.json` shows MCP vs API per tool. `tool_dream_mapping.json` maps each tool to dream allocation shifts. The demo is a visual representation of this architecture document.

17. Observability, Tracing & Monitoring

17.1 Tracing Architecture

Every agent run produces a complete trace. The OpenAI Agents SDK has built-in tracing that captures every step of the ReAct loop. We extend this with custom metadata for DFM-specific context.

Trace Field	Description	Example
trace_id	Unique ID for the entire request lifecycle	tr_abc123def456
agent_name	Which agent handled the request	Producer Agent
user_id	DFM team member who initiated	garrett@dragonfruitmedia.co
pipeline_phase	Which pipeline phase this relates to	Pre-Production
wishlist_tool	Which wish list tool was invoked	PR-4 (Claude-to-ClickUp)
steps[]	Array of reasoning + tool call steps	[{type: "reasoning", content: "..."}, {type: "tool_call", tool: "create_task", params: {...}}]
hitl_decisions[]	Array of HITL gate outcomes	[{gate: "Task Creation", decision: "approved", approver: "garrett", latency_sec: 45}]
handoffs[]	Array of cross-agent handoffs	[{from: "Producer", to: "PC", context: {...}}]
duration_ms	Total request duration	12450
tokens_used	Total LLM tokens consumed	{input: 2340, output: 890}
cost_usd	Estimated cost of this run	0.032

17.2 Monitoring Dashboards

Dashboard	Audience	Key Metrics
Agent Activity	DFM Leadership	Requests/day by agent, approval rate, avg response time, top tools used
HITL Performance	Producers / PCs	

		Approval latency, rejection rate, timeout rate, most-rejected actions
Cost Tracking	Finance	Daily/weekly/monthly API costs by agent, cost per request, cost per tool
Error Rates	Engineering / Ops	Error rate by tool, API failures, timeout events, handoff failures
Dream Allocation Progress	DFM Leadership	Time saved per role (estimated from agent usage), coordination overhead trend

17.3 Alerting Rules

Alert	Condition	Channel	Severity
High rejection rate	>30% of HITL requests rejected in 24h	#agent-ops	Warning
API failure spike	>5 consecutive failures for any connector	#agent-ops	Critical
Cost anomaly	Daily cost exceeds 2× rolling 7-day average	#agent-ops + Finance	Warning
HITL timeout spike	>20% of approvals timing out in 24h	#agent-ops	Warning
Sentiment drop	Client sentiment score drops below 0.4 (ED-5)	#agent-approvals-escalation	Critical

Architecture Summary — What This Document Delivers

Section	Deliverable	Evaluation Criteria Addressed
§1 Design Principles	6 principles grounded in DFM's specific workflow and constraints	30% Specificity
§2 Grand Schema	5-layer architecture diagram with interaction rules	25% Clarity of Architecture
§3 SDK Deep-Dive	Framework comparison, code examples, ReAct loop explanation	25% Clarity of Architecture
§4 MCP Decision Framework	Confirmed MCP availability for 8 systems, decision tree, hybrid binding	25% Clarity + 30% Specificity
§5 Agent Taxonomy	5 role agents + orchestrator with tools, phases, dream targets	30% Specificity
§6-10 Agent Details	Per-agent: config, tool definitions (code), ReAct flows, survey context	25% Clarity + 30% Specificity
§11 Handoff Architecture	8 handoff patterns with implementation code and end-to-end example	25% Clarity
§12 Data Flow	10-stage request lifecycle with diagram	25% Clarity
§13 HITL Gates	7 critical gates with implementation, graduation path	20% Roadmap + 30% Specificity
§14 Security	Auth per system, permission model per agent, error handling	25% Clarity
§15 Infrastructure	Deployment stack, container architecture	15% Cost Transparency
§16 Demo Mode	Mock data architecture aligned with React demo plan	25% Clarity
§17 Observability	Tracing schema, monitoring dashboards, alerting rules	20% Roadmap

Key Differentiator: This is not a generic architecture document. Every agent, every tool, every HITL gate, and every handoff pattern is mapped to DFM's specific pipeline phases, wish list tool IDs (CD-1 through CD-9, ED-1 through ED-5, PR-1 through PR-4), time audit percentages, and survey data. An engineer could begin implementation from this document tomorrow.